

SECCIÓN 1.2

DESARROLLO WEB ENTORNO **SERVIDOR** 2º DAW



CIPFP AUSIÀS MARCH
CENTRE INTEGRAT PÚBLIC
DE FORMACIÓ PROFESSIONAL



INTRODUCCIÓN A NODE.JS

Fundamentos, Entorno de Ejecución y Arquitectura Web

Asignatura: Desarrollo Entorno Servidor (DEWS)

Visión general del runtime y su rol en el desarrollo Backend



¿QUÉ ES NODE.JS?

- Definición clave: Es un runtime (entorno de ejecución) de JavaScript.
- Aspectos fundamentales:
 - Permite ejecutar código JavaScript fuera del navegador.
 - Se ejecuta en el servidor o en cualquier máquina local.
 - Funciona como un lenguaje backend completo (similar a Python, Java o PHP).
 - Takeaway principal: Convierte a JavaScript en un lenguaje de propósito general.

FLUJO CLIENTE-SERVIDOR Y PETICIONES WEB



- **Modelo Request / Response:**
 - Cliente (Browser/App): Realiza una petición (Request) enviando una URL/datos.
 - Servidor (Node.js): Procesa la petición, consulta datos y genera una respuesta (Response).
- **Tipos de Respuesta:**
 - Páginas HTML (con contenido dinámico).
 - Archivos estáticos (CSS, JS cliente, imágenes).
 - Datos puros (formatos JSON o XML para APIs).

¿POR QUÉ EJECUTAR CÓDIGO EN EL SERVIDOR?



- Acceso a Bases de Datos: Lectura y escritura segura de información persistente.
- Lógica de Negocio: Algoritmos pesados o privados que no deben exponerse al público.
- Seguridad y Autenticación:
 - Control de acceso y gestión de usuarios en un entorno no manipulable.
 - Validación de datos de entrada (protección contra manipulación del frontend).

NODE.JS VS. OTROS SERVIDORES WEB

■ Diferencia Técnica Clave:

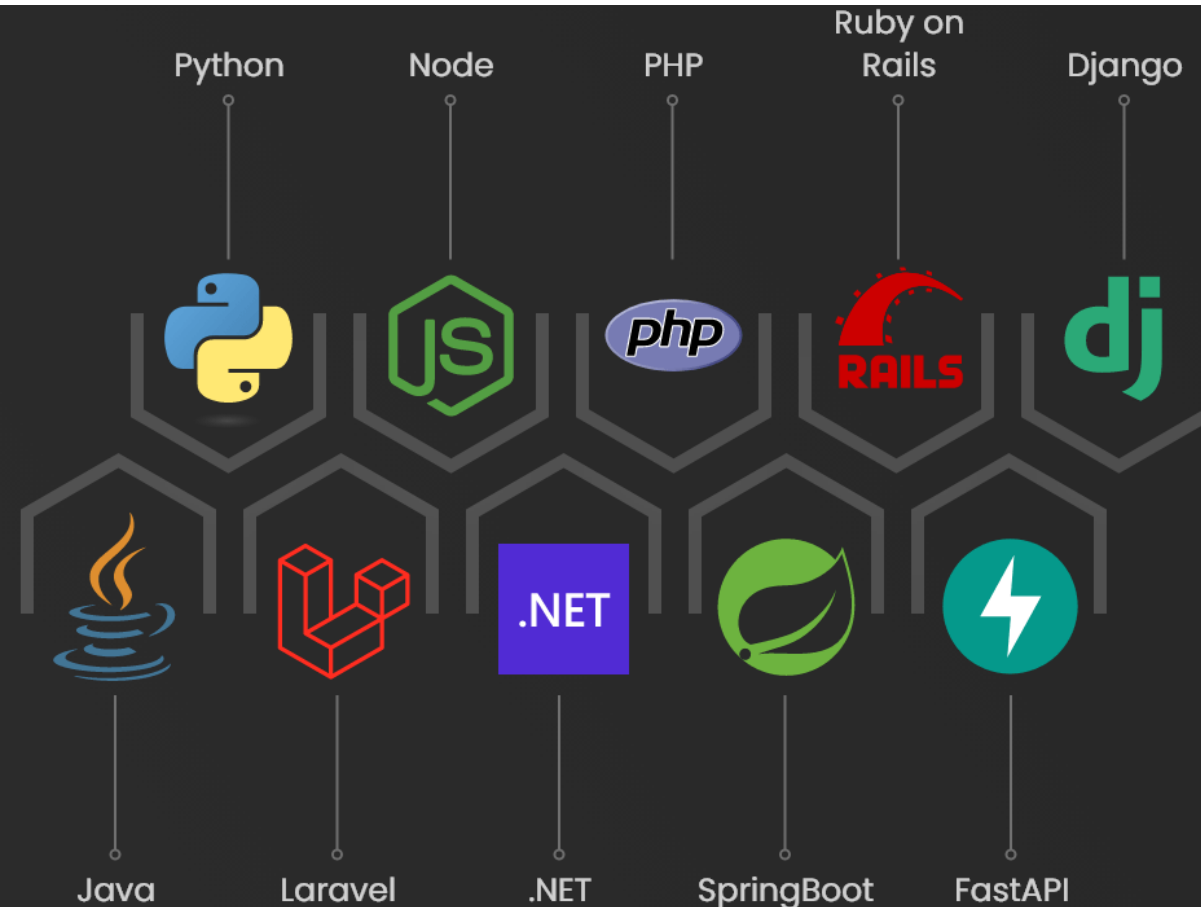
- Servidores tradicionales (ej. PHP): Requieren un servidor web externo (Apache, Nginx) para escuchar peticiones y delegar el código.
- Node.js: El propio código crea y gestiona el servidor web (escucha peticiones y las procesa en el mismo entorno).

■ Ecosistema y Alternativas Backend:

- Alternativas: Python (Django/Flask), PHP (Laravel), Java (Spring), C# (.NET).
- Ventaja de Node.js: Permite usar un único lenguaje (JavaScript) en toda la pila tecnológica (Frontend y Backend).

PRINCIPALES FRAMEWORKS

Top Backend Frameworks



***Framework** es un conjunto de herramientas, estructuras y reglas ya preparadas que te ayudan a desarrollar la parte del servidor de una aplicación web **sin tener que construirlo todo desde cero.**

USOS DE NODE.JS MÁS ALLÁ DEL SERVIDOR

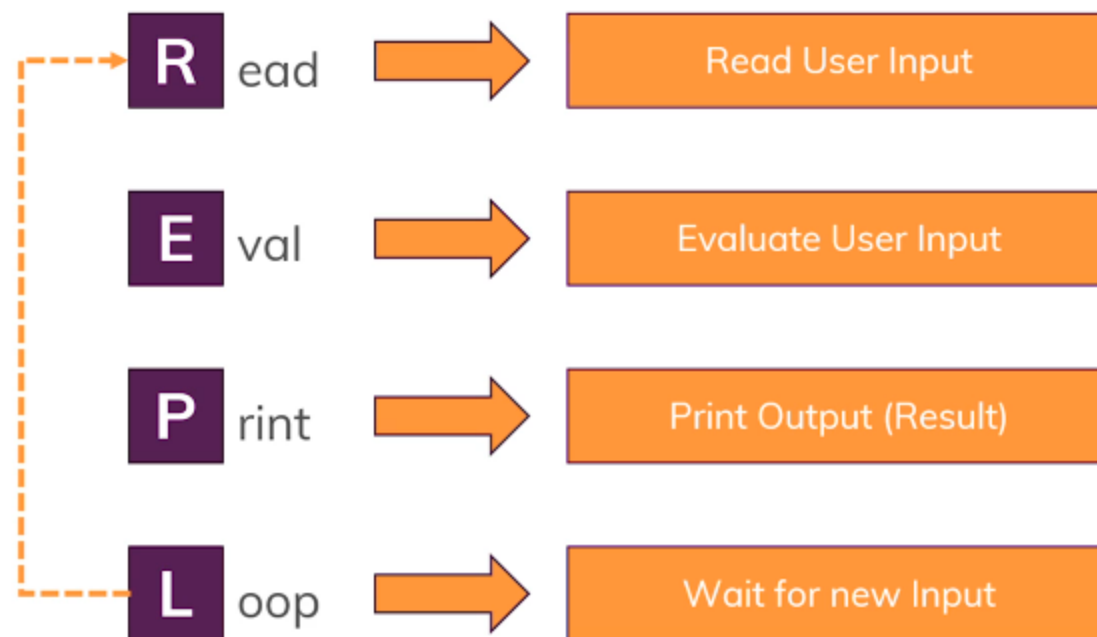


- No solo sirve para páginas web: Es un runtime de JS de propósito general.
- Herramientas de construcción y CLI:
 - Scripts de utilidad local y automatización de tareas.
 - Fundamento del ecosistema de compilación en frameworks Frontend (React, Angular, Vue).
- Acceso al File System: Creación de herramientas de línea de comandos para manipular archivos y flujos localmente.

FORMAS DE EJECUTAR NODE.JS: REPL VS. ARCHIVOS

REPL (Read-Eval-Print Loop):

- Read (Lee la entrada) → Eval (Evalúa el código) → Print (Imprime el resultado) → Loop (Bucle de espera).
- Se activa escribiendo node en la terminal (indicado por el símbolo >).
- Uso: Entorno de pruebas ("playground") para probar líneas de código en tiempo real.
- Inconveniente: El código no se guarda; al cerrar la terminal (Ctrl + C), se pierde.



FORMAS DE EJECUTAR NODE.JS: REPL VS. ARCHIVOS

Archivos de código (Script mode):

- Se escribe el código en archivos .js y se ejecutan con `node archivo.js`.
- Es el estándar para **aplicaciones reales**: permite guardar, compartir, desplegar y mantener el código.

Execute Files

Used for real apps

Predictable sequence of
steps

Use the REPL

Great playground!

Execute code as you write it

HOJA DE RUTA DEL DESARROLLO BACKEND



- Fundamentos y Express.js: Manejo de rutas, peticiones HTTP y motores de plantillas HTML.
- Persistencia de Datos: Conexión y gestión con bases de datos SQL (MySQL) y NoSQL (MongoDB).
- Seguridad y Flujos: Autenticación de usuarios, cookies, sesiones y validación de entradas.
- Servicios Avanzados: Carga/descarga de archivos, generación de PDFs, pagos y envío de emails.
- APIs Modernas: Construcción de REST APIs, GraphQL y comunicación en tiempo real con WebSockets.

*Una **API** (*Application Programming Interface*, o **Interfaz de Programación de Aplicaciones**) es una forma de permitir que **dos programas se comuniquen entre sí**, es decir, permite que un cliente (navegador, app móvil, otro servidor...) haga peticiones a nuestro servidor y reciba datos

JS EN EL SERVIDOR

JavaScript on the Server

